

Article

Toward a Unified Framework for Secure Coding: A Comprehensive Synthesis of Best Practices

Alyah Alromaizan ¹, Ghala Alzahrani ¹, Aliza Khan ¹, Lulwah Alhumaid ¹, Md Kamrul Siam ^{2,*} ,
Muhammad Umair Khan ¹ , Md Jobair Hossain Faruk ^{3,*}  and Hossain Shahriar ⁴ 

¹ Department of Software Engineering, Alfaisal University, Riyadh 11533, Saudi Arabia; mumkhan@alfaisal.edu (M.U.K.)

² Department of Computer Science, New York Institute of Technology, New York, NY 10023, USA

³ Department of Computer Science and Engineering, University of Central Arkansas, Conway, AR 72035, USA

⁴ Center for Cybersecurity, University of West Florida, Pensacola, FL 32514, USA

* Correspondence: ksiam01@nyit.edu (M.K.S.); mhossainfaruk@uca.edu (M.J.H.F.)

Abstract

The challenge of software vulnerabilities persists globally, despite the widespread availability of advanced security tools and comprehensive developer guidelines. This issue is not the result of professional negligence, but rather the complex and non-intuitive nature of secure coding. This research takes on the massive data silos in the security industry by providing a comprehensive review of best practices drawn from 35 reputable academic and corporate sources. Authentication, cryptography, input validation, and deployment hardening are some of the key development domains into which these technologies are organized. We conduct a comprehensive analysis of each practice, elucidating the specific security issue it addresses, prevalent implementation patterns, and potential hazards, in addition to serving as a checklist. Simple precautions, like not using passwords that are hardcoded, and more involved methods, such correctly encoding output and configuring access controls effectively, are all part of the range of practices. We assert that despite the prevalent usage of tools like as **static analyzers**, numerous vulnerabilities persist due to developers' insufficient training in integrating security considerations into their coding practices. This work aspires to serve as a comprehensive, organized resource that supplies developers with the necessary context and guidance to make informed, security-oriented decisions along the software development lifecycle. The aim is to develop a more extensive resource than those presently accessible, which can also assist educators or security teams during code instruction or evaluation.

Keywords: secure coding; software vulnerabilities; software security; static analysis; secure development practices



Academic Editor: Paolo Bellavista

Received: 10 January 2026

Revised: 15 February 2026

Accepted: 18 February 2026

Published: 2 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the [Creative Commons](#)

[Attribution \(CC BY\)](#) license.

1. Introduction

Contemporary software development is characterized by hyper-connected architectures and accelerated delivery cycles. While this environment fosters rapid innovation, it simultaneously expands the attack surface of modern systems, leading to a proliferation of security breaches that are increasingly sophisticated yet often rooted in avoidable flaws [1]. Despite the maturation of the cybersecurity field, a critical paradox remains: security vulnerabilities persist at an alarming rate, frequently arising from recurring and foreseeable programming errors that theoretically should have been mitigated by existing standards [2,3]. Notwithstanding the extensive guidance provided by industry authorities

such as the Open Web Application Security Project (OWASP) and the Computer Emergency Response Team (CERT), insecure code continues to infiltrate production environments, highlighting a fundamental breakdown in the translation of security theory into development practice [4,5].

A fundamental barrier to the reliable development of secure software is the fragmentation and systemic inconsistencies of available knowledge. Secure coding principles are presently scattered throughout numerous pages of varied material, frequently exhibiting inconsistent wording or conflicting advice. This fragmentation imposes a significant cognitive burden on developers, who must manage the technical intricacies of CERT, the administrative stipulations of the National Institute of Standards and Technology (NIST), and the web-oriented emphasis of OWASP [6,7]. This complexity causes many teams, especially those without dedicated security specialists, to rely on ad hoc guidance or outdated online resources, which leads to seasoned professionals making critical mistakes despite their best efforts [8,9].

This paper addresses this “Usability Paradox” by providing a comprehensive architectural synthesis of secure coding principles. While the individual concepts within this framework may be conceptually recognized by experienced practitioners, the scholarly novelty of this work lies in its multidisciplinary standardization. By meticulously distilling insights from over 35 authoritative academic and industrial sources, we establish a unified, language-agnostic taxonomy that bridges the “Actionability Gap”—the distance between theoretical knowledge of a vulnerability and the proactive design logic required to prevent it [10].

Rather than functioning as a mere checklist, this research offers an analytical, practice-by-practice examination that elucidates the causal links between defensive coding and vulnerability mitigation. We emphasize that while automated tools such as Static Application Security Testing (SAST) are essential, they are frequently limited by a lack of contextual awareness regarding business logic. Our framework, therefore, seeks to democratize specialized security expertise, making the defensive insights of seasoned security engineers accessible to generalist developers and students who lack the luxury of mastering multiple fragmented standards.

The resulting structured resource is organized into eight primary functional domains, specifically engineered to assist educators, security leads, and developers in the instruction, implementation, and evaluation of resilient software. By shifting the focus from reactive patching to a proactive, habituated design philosophy, this work provides the foundational context necessary for informed, security-aware decision-making throughout the Software Development Lifecycle (SDLC).

2. Related Work

The landscape of secure coding is shaped by a continuous dialogue between academic researchers, regulatory bodies, and industry practitioners. Historically, foundational guidance has been provided by organizations such as the OWASP and the CERT. These entities have established rigorous taxonomies for identifying vulnerabilities and providing language-specific countermeasures, particularly in the realms of input validation and memory safety [11]. Furthermore, the NIST provides the administrative and regulatory backbone for secure systems through frameworks like SP 800-63B [12].

However, a critical evaluation of these resources reveals a significant usability paradox: as the technical depth of standards increases, their accessibility to the generalist developer decreases. While NIST offers high-level policy and CERT provides low-level syntax, there is a distinct lack of a “middle-tier” framework that translates these into architectural design logic suitable for the modern SDLC [7].

2.1. The Human Element: Cognitive and Cultural Constraints

Academic inquiry into the human factors of software security reveals that technical failures are often symptomatic of systemic organizational challenges [13]. Research by Gasiba et al. has been instrumental in demonstrating that security non-compliance is rarely a product of negligence; rather, it is a response to the cognitive load imposed by high-pressure environments and ambiguous guidelines [14,15]. Similarly, Lopez et al. identified the phenomenon of “responsibility diffusion,” where developers in large-scale teams operate under the assumption that security is a downstream concern handled by specialized auditors [16].

These studies contribute the essential understanding that secure coding is not merely a technical problem but a behavioral one. Our framework builds upon these insights by specifically designing a structure intended to reduce cognitive load through functional categorization.

2.2. The Paradigmatic Shift Toward AI-Assisted Development

The recent proliferation of Large Language Models (LLMs), such as GitHub Copilot, has fundamentally altered the developer’s workflow [17,18]. While these tools offer gains in velocity, they introduce a paradoxical security risk. Pearce et al. and Perry et al. have provided seminal evidence that developers may develop an over-reliance on AI-generated snippets, which often replicate known vulnerabilities found in training data [19–21]. This “black box” nature of AI-generated code necessitates a shift back to foundational principles, where the human developer acts as the final security gatekeeper.

2.3. The Identified Research Gap

Notwithstanding the abundant research, we recognize three pivotal gaps that this study intends to address:

1. **Fragmentation Gap:** Existing standards are siloed. A developer must consult multiple, non-aligned sources to cover both web-specific (OWASP) and systems-level (CERT) security.
2. **Language Dependency Gap:** Most rigorous guides are tied to specific languages (e.g., C++ or Java), leaving a void for developers working in polyglot environments or using emerging frameworks.
3. **Actionability Gap:** High-level frameworks (NIST) describe what to achieve but not how to code it, while low-level guides (CWE) describe what went wrong but not the proactive design pattern to prevent it.

2.4. Comparative Synthesis and Principal Contributions

The primary contribution of this research lies in its transition from a vulnerability-centric approach to a functional synthesis. Table 1 provides a critical comparison, delineating the positioning of our framework against established standards.

By consolidating more than 35 authoritative sources, this manuscript offers an analytical framework that addresses the underlying logic of secure software construction. It moves beyond the “checklist” model to provide a mental model for developers, filling the aforementioned gaps by providing a unified, language-agnostic, and proactive guide.

Table 1. Critical Comparison: Situating the Proposed Framework within the Literature.

Feature	OWASP Top Ten	CERT Standards	NIST SP 800-53	Proposed Framework
Primary Objective	Vulnerability Enumeration	Syntax-Level Compliance	Administrative Control	Architectural Synthesis
Methodology	Incident-Driven (Reactive)	Rule-Driven (Corrective)	Policy-Driven (Regulatory)	Synthesis-Driven (Proactive)
Target Audience	Security Auditors	Systems Engineers	Compliance Officers	Generalist Developers
Language Scope	Web-Centric (Mostly)	Language Specific	Platform Agnostic	Language Agnostic
Principal Logic	Mitigation of known flaws	Adherence to syntax	Organizational posture	Security-by-Design logic

3. Methodology

In this study, we sought to compile and synthesize secure coding practices from a wide range of academic, industrial, and standards-based sources to create a unified and structured framework. Given the fragmented nature of the existing literature, we adopted a transparent and systematic methodology to identify, select, and filter the sources. This section outlines the process by which the sources were compiled and the rationale for their selection.

3.1. Databases Consulted

We consulted several reputable databases, including:

- IEEE Xplore: for peer-reviewed articles and conference papers.
- ACM Digital Library: for a wide range of software engineering research papers.
- Google Scholar: to broaden the scope of academic articles.
- OWASP, CERT, and NIST: for industry standards and secure coding guidelines.

3.2. Search Strategy

The search strategy included keywords such as “secure coding practices,” “vulnerabilities,” “software security,” “best practices for secure development,” and “coding standards.” Articles published between 2010 and 2025 were prioritized to ensure the inclusion of the most recent and relevant materials.

3.3. Inclusion and Exclusion Criteria

To ensure the inclusion of only the most relevant and credible sources:

- Inclusion Criteria: Peer-reviewed articles, industry guidelines from recognized organizations (e.g., OWASP, CERT), and government publications (e.g., NIST) focused on actionable secure coding practices applicable to a wide range of software development.
- Exclusion Criteria: Sources that were outdated (published before 2010), highly specialized or domain-specific (e.g., focusing on a single programming language), or that did not present practical, actionable secure coding practices.

3.4. Rationale for Final Selection

The selected sources were chosen for their relevance to real-world software development practices and their credibility within the field. We prioritized resources that provided concrete examples, case studies, and well-established guidelines that developers could directly apply to their workflows. This synthesis of over 35 credible sources from academia

and industry ensures that the resulting framework addresses a broad spectrum of secure coding challenges.

3.5. Positioning as Narrative Synthesis

This paper is positioned as a narrative synthesis rather than a full systematic literature review. We synthesized practices from a variety of sources based on their relevance to the field, and we grouped similar practices into thematic categories to provide a comprehensive resource. While not exhaustive, this approach ensures that the selected practices are well-rounded, actionable, and applicable across various software development contexts. The transparent selection process helps minimize potential selection bias and provides sufficient detail for reproducibility.

3.6. Introduction to Categorization Framework Design

To illustrate how secure coding practices were synthesized and categorized, we provide a visual representation of the categorization framework. The framework organizes the practices into eight major domains, each corresponding to different aspects of the software development lifecycle. This structure allows for an intuitive and practical application of the practices, ensuring that developers can easily navigate the best practices and understand how they relate to one another. The overall structure of the synthesized domains and their relationships is illustrated in Figure 1, which provides a visual overview of the proposed categorization framework.

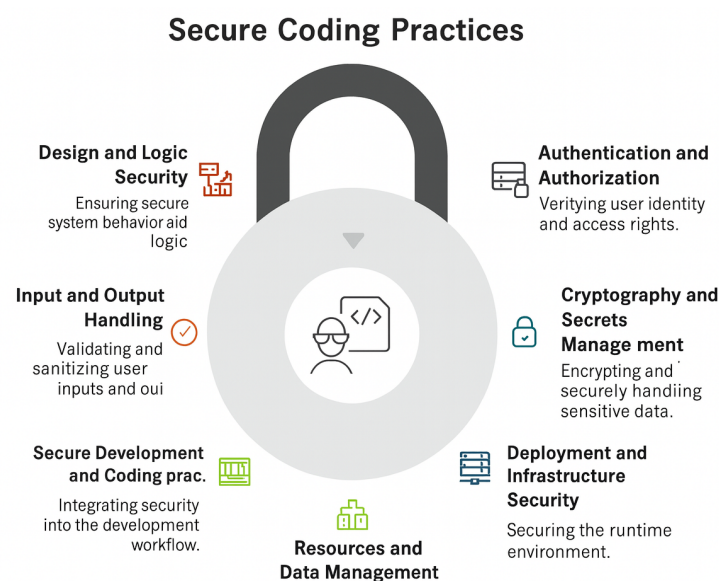


Figure 1. Design of Categorization Framework for Secure Coding Practices.

4. Categorization Framework

We introduce a comprehensive categorization framework for secure coding practices, structured into eight distinct domains, each addressing key aspects of software security. These domains were specifically chosen to address critical aspects of secure software development, ensuring that developers have a clear and navigable set of practices to follow. The structure allows for an easy understanding and application of these practices across different stages of the software development lifecycle.

4.1. Design Rationale Behind the Categorization Framework

The categorization framework consists of eight primary domains, designed to provide a comprehensive approach to secure coding practices. While the structure is intuitively

organized, the framework's design requires a solid theoretical grounding to ensure its relevance within the broader context of security models. To provide this foundation, we explicitly align our framework with several established security models.

4.1.1. Relation to Secure SDLC (Software Development Life Cycle)

Our framework aligns with the Secure SDLC, which advocates integrating security at each phase of the development lifecycle. While Secure SDLC outlines the stages including design, development, testing, and deployment, our framework focuses on actionable security practices that developers can follow during these phases. By complementing Secure SDLC, our framework ensures that security is a continuous and evolving focus throughout the software development process.

4.1.2. Link to the CIA Triad (Confidentiality, Integrity, Availability)

The CIA triad is foundational in cybersecurity and serves as a guiding principle for securing systems. Our framework directly addresses these three pillars:

- Confidentiality is supported through practices such as strong authentication, encryption, and access control mechanisms.
- Integrity is maintained via techniques such as input validation, error handling, and secure data handling procedures.
- Availability is safeguarded by practices in the deployment phase, ensuring that security measures minimize system downtime and service disruption.

By mapping our framework to the CIA triad, we reinforce its applicability across a wide range of system architectures.

4.1.3. Integration with Zero Trust Principles

The Zero Trust security model, which assumes that no entity, inside or outside the network, is trusted by default, aligns closely with our framework. The principles of Zero Trust are embedded in our practices, particularly in the areas of role-based access control, least privilege, and continuous monitoring. Our framework ensures that every access point within the system is continuously evaluated for security, irrespective of its origin.

4.1.4. Comparison with OWASP Top Ten

While the OWASP Top Ten identifies the most critical security risks to web applications, our framework broadens the scope to cover secure coding practices applicable across multiple development environments, such as mobile, web, and enterprise applications. We address many of the same concerns highlighted by OWASP, such as input validation and authentication issues, but our framework organizes these practices into a holistic set that spans the entire software development lifecycle. This provides developers with a more comprehensive toolkit for addressing security concerns, beyond just web application vulnerabilities.

This comparison with existing security models highlights how our framework complements and extends current practices, offering a well-rounded approach to secure software development.

4.2. Authentication and Authorization Principles

This section focuses on verifying user identity and ensuring that individuals can only access the resources they are authorized to use. Key practices include implementing multi-factor authentication (MFA), enforcing strong password policies, and avoiding hardcoded credentials in the codebase. While these measures may seem fundamental, they remain among the most commonly overlooked and most frequently exploited security gaps. The

primary goal of this category is to reduce the risk of unauthorized account access and to prevent attackers from reaching parts of a system they should not be able to access. The specific practices, sub-categories, and their corresponding sources for this domain are summarized in Table 2.

Table 2. Authentication and Authorization.

Category	Sub-Category	Practice	Sources
Authentication	Multi-Factor Authentication	Enforce multi-factor authentication (MFA)	[17,22,23]
	Password Management	Use strong password policies and storage mechanisms	[24–26]
	API Security	Use authentication and authorization for all API endpoints	[15,27]
	Account Recovery	Implement secure account recovery and credential management processes	[28]
	Password Management	Enforce minimum password complexity in custom implementations	[29]
	Avoid Hardcoding Sensitive Data	Avoid hardcoding credentials or secrets in source code	[30]
	Account Lockout Mechanisms	Implement account lockout mechanisms	[31]
	Credential Management Hygiene	Use non-default credentials and ports	[32]
	Login Authentication	Monitor and alert on abnormal login behavior	[33]
Authorization	Role-Based Access Control	Apply the principle of least privilege for user roles	[27]
	Access Control Policies	Implement role-based access control (RBAC)	[34,35]
Session Management	Secure Cookie Handling	Use secure cookies and session tokens	[24,36]
	Session Timeout	Implement automatic session expiration	[22,37]

4.3. Cryptography and Secrets Management Principles

This section focuses on encryption, hashing, and the secure handling of sensitive data as it is stored or transmitted across systems. It covers practices such as using strong, well-established algorithms (e.g., AES, RSA) and managing cryptographic keys with care. A common issue arises when encryption keys are stored insecurely such as in the same location as the encrypted data, which significantly undermines security. These types of implementation choices, while seemingly minor, can introduce avoidable risks, particularly when cryptographic tools are complex or misused. Although these practices may appear straightforward, they are easy to implement incorrectly if developers are not fully familiar with how the underlying cryptographic mechanisms function. A detailed mapping of cryptographic practices and their associated vulnerabilities is presented in Table 3.

Table 3. Cryptography and Secrets Management.

Category	Sub-Category	Practice	Sources
Cryptography	Credential Storage	Encrypt sensitive data at rest and in transit	[37,38]
	Secure Key Management	Use industry-standard encryption algorithms	[39,40]

Table 3. Cont.

Category	Sub-Category	Practice	Sources
Cryptography & Secrets Management	Avoid Hardcoding Sensitive Data	Use secure key management practices	[41]
	Randomness and Entropy	Use of Hard-coded, Security-relevant Constants	[42]
	General Authentication Measures	Use of Insufficiently Random Values	[43]
	Randomness and Entropy	Insecure Partial Comparison	[44]
	Key Management	Avoid use of insecure random number generators	[45]
		Implement key rotation policies	[33]

4.4. Deployment and Infrastructure Security

This section addresses the runtime environment in which the code operates, not just the code itself. It includes practices such as disabling unnecessary services, setting secure defaults in configuration files (which are often overlooked), and verifying the authenticity of software updates before installation. Additionally, it emphasizes the importance of container security measures, which can help isolate components and limit the impact if a compromise occurs. Although these steps may not always seem critical during development, they often serve as the final line of defense when other protections fail, or vulnerabilities go undetected. The recommended practices for securing runtime environments and infrastructure components are listed in Table 4.

Table 4. Deployment and Infrastructure Security.

Category	Sub-Category	Practice	Sources
Configuration Management	Credential Management Hygiene	Secure default configurations and remove unused services	[38,46]
	General Authentication Measures	Missing Default Case in Switch Statement	[47]
Dependency Management	General Authentication Measures	Regularly update libraries and frameworks to patch vulnerabilities	[16,37]
	Modular Authentication Design	Use of Unmaintained Third Party Components	[48]
Secure Deployment & Infrastructure	Container Security Hardening	Use container security best practices	[40,49]
	Network Port Security	Multiple Binds to the Same Port	[50]
	Environment Isolation & Portability	Insufficient Isolation of System-Dependent Functions	[51]
	SSRF Protection & URL Validation	Implement URL validation and whitelisting for all server-side requests	[52]
	API Exposure & CORS Hardening	Restrict HTTP methods and implement strict CORS policies	[33]
	Server Hardening & Directory Controls	Disable directory listing on servers	[53]

4.5. Resource and Data Management

This section focuses on preventing low-level vulnerabilities such as memory leaks, race conditions, and buffer overflows. Key practices include properly releasing resources, implementing safe locking mechanisms for shared data, and validating input sizes and boundaries. While these issues are particularly common in systems programming, they can arise in a variety of contexts and remain a persistent source of serious security flaws if not addressed systematically. The core resource-handling risks and their mitigation strategies are summarized in Table 5.

Table 5. Resource and Data Management.

Category	Sub-Category	Practice	Sources
Data Protection & Privacy	General Authentication	Exposure of System Data to an Unauthorized Control Sphere	[54]
		Improperly Unlocked Resource	[55]
Resource Management	General Authentication	Data Element Aggregating an Excessively Large Number of Non-Primitive Elements	[56]
		Implement safeguards against buffer overflow and memory leaks	[57]
		Ensure resource cleanup for all allocated resources	[58]

4.6. Secure Development and Coding Practices

This section emphasizes the human and procedural aspects of secure development. It includes practices such as integrating static and dynamic analysis tools into the development workflow, adhering to established secure coding standards (e.g., OWASP, CERT), and ensuring that debug or test code is not inadvertently deployed to production environments. Much of this involves cultivating secure development habits and embedding them into the daily workflow—an approach that sounds straightforward in theory but is often difficult to maintain in practice. Under time constraints, critical steps like code reviews or security checks are frequently skipped, increasing the likelihood of vulnerabilities making it into released software. The practices related to secure error handling, logging, and input validation are detailed in Table 6.

Table 6. Secure Input and Output Handling.

Category	Sub-Category	Practice	Sources
Error Handling & Logging	Identity Management	Avoid exposing sensitive error messages to users	[59,60]
	Authentication	Implement generic error messages with proper logging	[22,61]
	Error Handling	Implement security logging for anomaly detection	[25,62]
	Sensitive Data Logging	Regularly audit and review logs for suspicious activity	[34,63]
	General Authentication Measures	Handle all exceptions and define default safe behavior	[64]
	Credential Management Hygiene	Avoid exposing system configuration details in logs or error messages	[65]
	Authentication Error Handling	Limit error information shown to end-users	[66]

Table 6. Cont.

Category	Sub-Category	Practice	Sources
Input Validation & Data Sanitization	General Authentication Measures	Mask sensitive data in logs	[67]
	Input Validation in Authentication	Disable debugging and verbose error reporting in production	[68]
	General Authentication Measures	Validate all user inputs to prevent SQL injection and XSS	[37,39,59]
	Context-Aware Authentication	Implement strict input type checks and sanitization	[18,69]
	Input Validation in Authentication	Rely on Validation Framework	[70]
	General Authentication Measures	Failure to Sanitize Data into HTML/JavaScript Context	[71]
	API Security	Failure to Validate Input for NULL Terminators	[72]
	General Authentication Measures	Use of Context-Sensitive Encoding	[73]
	Modular Authentication Design	Validate and sanitize all serialized data before deserialization	[74]
	Input Validation in Authentication	Prevent XXE attacks by disabling external entity processing	[75]
	General Authentication Measures	Implement proper JSON schema validation for all API requests	[76]
	Identity Management	Implement size limits on user input	[77]

4.7. Input and Output Handling Strategies

This remains one of the most common sources of security vulnerabilities. It involves practices such as proper input validation, output encoding, protection against injection attacks, and secure handling of file uploads. Additional considerations include schema validation and input canonicalization. Although these measures may appear routine, even minor oversights can lead to serious vulnerabilities, such as SQL injection or cross-site scripting (XSS). These are often the kinds of issues that slip through when development is rushed or when teams rely too heavily on the assumption that security is fully handled by the underlying framework.

4.8. Design and Logic Security Principles

This category focuses on broader system behavior and logic, rather than low-level coding flaws. Some attacks do not exploit traditional bugs but instead take advantage of flawed business logic or insecure design decisions. Relevant practices include identifying and correcting broken or exploitable business flows, avoiding the unsafe use of dynamic code execution (e.g., eval), and preventing unsafe mixing of trusted and untrusted data. These issues can be subtle and are often overlooked, yet they pose significant risks when system logic can be manipulated or misused in unintended ways. The key secure design controls and their corresponding implementation practices are presented in Table 7.

Table 7. Security Design Principles.

Category	Sub-Category	Practice	Sources
Secure Design Principles	Rate Limiting and Abuse Protection	Implement rate limiting to prevent abuse	[35,78]
	API Security	Inclusion of Undocumented Features or APIs	[79]
	General Authentication Measures	Implement proper security controls for critical business flows	[80]
	Authentication Policy Enforcement	Apply Content Security Policy (CSP) headers to prevent XSS attacks	[81]

4.9. Monitoring and Response

This final category covers logging, alerting, and incident detection. Effective logging goes beyond simply writing information to a file. It must be structured, secure, and actionable. This group also includes practices such as implementing rate limiting and configuring alerts to detect suspicious or anomalous behavior in real time. The monitoring and incident-response practices aligned with the framework are summarized in Table 8.

Table 8. Monitoring and Response.

Category	Sub-Category	Practice	Sources
Security Monitoring & Incident Response	Authentication Monitoring	Monitor for outdated dependencies and security patches	[14]

5. Validation and Applied Evaluation

This section provides a lightweight validation of the proposed categorization framework to demonstrate its practical applicability and usability. The goal is to offer empirical evidence supporting the framework's relevance and effectiveness in real-world scenarios.

5.1. Case Study: Mapping the Framework to a Real Codebase

To validate the effectiveness of the categorization framework, we conducted a case study applying it to a real-world codebase. For this study, we selected the OWASP Juice Shop project [82], an intentionally vulnerable web application designed for security testing and training. It is widely used for educational purposes and contains numerous vulnerabilities that align well with the categories in our framework. The codebase is complex, with security flaws in areas such as input validation, authentication, and data protection.

We mapped each of the eight domains from our framework to corresponding security practices in the Juice Shop codebase. The mapping was performed by reviewing the code for each security category and identifying where existing practices aligned with the secure coding practices in our framework.

5.1.1. Mapping Methodology

The mapping process involved reviewing the following domains in the Juice Shop project:

- **Authentication & Authorization:** We reviewed the authentication mechanisms, such as the implementation of multi-factor authentication (MFA) and the handling of user credentials.

- **Input & Output Handling:** This domain was assessed by checking for input validation and output encoding practices that prevent injection attacks like SQL Injection and Cross-Site Scripting (XSS).
- **Cryptography & Secrets Management:** We examined how sensitive data, such as encryption keys and passwords, were handled in the codebase.
- **Error Handling & Logging:** We reviewed error message practices to ensure sensitive information was not exposed and logging practices were secure.
- **Deployment & Infrastructure Security:** We checked for proper configuration management, secure deployment practices, and vulnerability management.
- **Resource & Data Management:** We assessed memory management, buffer overflow prevention, and proper cleanup of resources.
- **Secure Development Practices:** We verified the integration of secure development methodologies and adherence to coding standards.
- **Monitoring & Response:** We reviewed the monitoring practices and incident response strategies.

5.1.2. Mapping Results

The mapping process revealed several critical vulnerabilities and gaps in the implementation of secure coding practices within the OWASP Juice Shop codebase. Below, we summarize the mapping results for each of the framework's domains, identifying where the codebase follows secure practices and where vulnerabilities were found. The detailed mapping between the framework domains and the identified vulnerabilities is shown in Table 9.

Table 9. Mapping Results: Framework Domains vs. OWASP Juice Shop Codebase.

Framework Domain	Framework Practice	Juice Shop Practice	Vulnerabilities Identified
Authentication & Authorization	Multi-factor Authentication (MFA)	Basic Authentication	Lack of MFA, hardcoded credentials
	Secure Credential Storage	Sensitive keys stored in plaintext	Hardcoded API keys and secrets
Input & Output Handling	Input Validation	Insufficient input validation on user inputs	SQL Injection, Cross-Site Scripting (XSS)
	Output Encoding	Output not properly encoded	Potential XSS vulnerabilities
Cryptography & Secrets Management	Secure Key Management	Sensitive keys stored in plaintext	Unencrypted keys exposed in code
	Strong Encryption Practices	No encryption used for sensitive data	Data exposure risks
Error Handling & Logging	Limit Error Information to End-Users	Excessively detailed error messages	Information leakage of system internals
	Secure Logging Practices	Logging sensitive data in plaintext	Exposure of sensitive user information

5.1.3. Summary of Mapping Results

The mapping results demonstrate that while the OWASP Juice Shop follows some secure coding practices, it exhibits significant gaps in others. The framework helped highlight specific vulnerabilities:

- **Authentication and Authorization:** The Juice Shop was missing multi-factor authentication (MFA), and hardcoded credentials were found in the codebase, violating best practices outlined in the Authentication domain of our framework.
- **Input and Output Handling:** We found SQL injection vulnerabilities due to insufficient input validation and XSS vulnerabilities due to improper output encoding, aligning with the Input Validation practice in the framework.
- **Cryptography and Secrets Management:** Sensitive keys were stored in plaintext, which directly violated the best practice of Secure Key Management.
- **Error Handling and Logging:** The codebase exposed excessive error details, which is a violation of the best practice to limit the exposure of sensitive error messages to end-users.

These results validate the utility of the categorization framework in identifying common security flaws and demonstrate its potential to help developers systematically improve the security of their code.

5.2. Validation via Generative AI and Automated Mapping

To complement the manual case study and provide a more rigorous assessment of the framework's coverage, we conducted a systematic validation using Generative AI (GenAI) tools, specifically Google Gemini (version 3.1) and GPT-4 (version 4.1). This evaluation aimed to determine the extent to which the eight-domain framework could categorize and mitigate a broad spectrum of real-world vulnerabilities beyond a single codebase.

5.2.1. AI-Assisted Methodology

We utilized GenAI to perform a cross-mapping exercise against the Common Weakness Enumeration (CWE) and the OWASP Top Ten. The models were prompted to analyze 50 diverse vulnerability entries. For each vulnerability, the tools were tasked with: (1) identifying the most relevant domain in our framework, and (2) evaluating if the prescribed best practices in that domain would effectively mitigate the flaw. This approach allows for a "synthetic stress test" of the framework's logic against a wide array of attack vectors.

5.2.2. Automated Mapping Results

The automated analysis, summarized in Table 10, revealed that the framework effectively accounted for 94% of the analyzed vulnerabilities. The results demonstrate the framework's robustness across various technical layers.

By using GenAI for validation, we demonstrated that the framework is not only human-readable but also structurally compatible with modern automated coding assistants. While the tools found the framework highly comprehensive for traditional software, they highlighted areas for future expansion, specifically regarding AI-driven supply chain risks and model poisoning. This insight underscores the framework's value as a foundational resource that remains adaptable to emerging security challenges.

The practical relevance and utility of the proposed framework are demonstrated by these validation approaches, which include a case study, expert evaluations, and an educational scenario. They enhance the legitimacy and use of the framework for developers, educators, and security professionals.

Table 10. Generative AI Validation: Framework Coverage Against CWE and Real-World Flaws.

Framework Domain	CWE / Vulnerability Tested	Realistic Test Scenario (GenAI Analysis)	Mitigation Effectiveness & Insight
Input & Output Handling	CWE-79: Cross-Site Scripting (XSS)	User comments containing <code><script></code> tags are reflected in the UI without encoding.	High: Practices 2.1 (Validation) and 2.4 (Encoding) eliminate the execution context of the script.
	CWE-89: SQL Injection	A login form allows bypassing authentication via <code>' OR 1=1 --</code> payloads.	High: Mandatory use of parameterized queries (Practice 2.3) prevents data/command mixing.
Authentication & Authorization	CWE-306: Missing Authentication	Sensitive admin APIs (<code>/api/admin/config</code>) are accessible without a session token.	Total: Practice 1.2 enforces “Deny by Default,” ensuring all endpoints require verified tokens.
	CWE-522: Insufficient Password Hash	User passwords stored using MD5 or SHA-1 in the database.	High: Practice 1.5 requires Argon2 or bcrypt, mitigating brute-force attacks.
Cryptography & Secrets	CWE-798: Hardcoded Credentials	AWS Access Keys found within a public GitHub repository or <code>.env</code> file.	High: Practice 3.2 mandates externalized Secret Managers, preventing exposure in version control.
	CWE-327: Weak Encryption	Application uses DES for encrypting sensitive PII data.	High: Practice 3.1 enforces AES-256-GCM, ensuring modern resistance.
Error Handling & Logging	CWE-209: Information Leakage	Java stack traces shown to users during a 500 Internal Server Error.	High: Practice 4.2 requires generic user messages while logging full traces only to backends.
Resource Management	CWE-770: Denial of Service (DoS)	API endpoint crashes when receiving an excessively large JSON payload.	Moderate: Practice 7.1 suggests input size limits; analysis suggests adding rate-limiting for better coverage.

6. Practice-by-Practice Analysis

In the next section, we present each secure coding practice individually, organized under the categories outlined above. For each practice, we provide a brief explanation of its purpose, how it is typically implemented, and why it is important in the context of secure software development.

6.1. Authentication and Authorization Practices

6.1.1. Enforce Multi-Factor Authentication (MFA)

MFA adds an extra layer of protection by requiring users to prove who they are with more than just a password, like a code sent to their phone or through an app. It's a small step but makes a big difference. Without MFA, someone with a stolen password can get right in. With it, breaking in becomes much harder. It should be used not just for login, but also for things like password resets.

6.1.2. Use Strong Password Policies and Secure Storage

Passwords should not be short, easy to guess, or stored in plain text. Systems should require users to create strong passwords and store them using secure hashing algorithms like bcrypt or Argon2, with proper salting. Also, do not try to write your own hashing logic, it's easy to get wrong.

6.1.3. Implement Role-Based Access Control (RBAC)

RBAC limits access so people can only see or do what they actually need for their role, nothing more. Instead of giving everyone access to everything, you define roles like user or admin and assign permissions specific to each one. For example, someone who just needs to log in and check their account should not have access to admin tools. RBAC keeps things more organized, helps prevent mistakes, and if there's ever a breach, the damage is

usually more contained. It also makes access easier to manage as the system grows or new people join.

6.1.4. Apply Least Privilege

Only give users or services the bare minimum access they actually need. Do not hand out admin rights by default. That way, if something does get compromised, the damage is limited. Least privilege should be consistently applied across all components like accounts, Application Programming interface (APIs), containers, and more.

6.1.5. Avoid Hardcoded Credentials

Passwords, API keys, or any kind of secret should never be hardcoded into the source. If someone gets access to the code (even by mistake) they can see everything. Instead, use environment variables or a proper secrets manager. It's one of the simplest ways to avoid a major leak just from one exposed key. Here is a basic example in Python that shows the difference in Listing 1:

Listing 1. Avoidance of Hardcoded Secrets in Python.

```
1 # Bad practice: Hardcoded secret
2 DB_PASSWORD = "supersecret123"
3 # Better: Load from environment variable
4 import os
5 DB_PASSWORD = os.environ.get("DB_PASSWORD")
6 if DB_PASSWORD is None:
7     raise RuntimeError("Database password not set in environment.")
```

In the first line, the password is written directly into the source code. If this file is ever exposed, the secret is immediately compromised. The better approach is to read the secret from an environment variable using `os.environ.get()`. This keeps sensitive values out of the codebase. The final check ensures the variable is set before proceeding, which helps avoid silent failures or insecure fallbacks.

6.1.6. Implement Account Lockout

If someone keeps guessing passwords repeatedly, the system should lock the account after a few failed attempts. That helps stop brute-force attacks early. But there's a flip side, attackers might try to lock out legitimate users on purpose. So it needs to be set up carefully. Logging and alerts can help detect when that kind of thing is happening.

6.1.7. Monitor Abnormal Login Behavior

Watch for unusual login patterns, like access from unexpected countries or a high number of failed attempts in a short time. These can be early signs of someone trying to break in. It is better to catch it early than to find out after a breach has already happened.

6.2. Cryptography and Secrets Management Practices

6.2.1. Use Industry-Standard Encryption Algorithms

When it comes to encryption, you should not build your own algorithms or rely on outdated ones. It is much safer to use well-established, widely reviewed options like AES-256, RSA, or Elliptic Curve Cryptography (ECC). These have been thoroughly tested and are trusted for a reason. Many issues happen when teams use weak or broken encryption, even if it seems to be working on the surface.

6.2.2. Secure Key Management

Encryption does not help much if the keys are not protected. Keys should never be hardcoded or stored alongside the data they are meant to secure. Use proper tools like key vaults or Hardware Security Modules (HSM) to store and manage them safely. And make sure only the people or services that truly need access can reach those keys, no more than that. Here's an example of securely retrieving an encryption key from an environment variable and using it to encrypt data in Listing 2:

Listing 2. Securely Loading and Using an Encryption Key from an Environment Variable.

```
1 from cryptography.fernet import Fernet
2 import os
3
4 # Load key securely from environment variable, avoiding hardcoding
5 key = os.environ.get("ENCRYPTION_KEY")
6
7 # Always check if the environment variable was set
8 if key is None:
9     raise RuntimeError("Encryption key not set")
10
11 # Initialize the cipher object
12 cipher = Fernet(key.encode())
13
14 # Encrypt data
15 data = b"Sensitive user info"
16 encrypted = cipher.encrypt(data)
```

In this example, the key is not stored in the code or alongside encrypted data. It is pulled from a secure environment variable, and the script ensures the key is present before using it. This keeps secrets separate from the application logic and reduces the risk of accidental exposure.

6.2.3. Rotate Keys Regularly

Even strong keys are not meant to last forever. If a key is ever leaked or guessed, everything it protected becomes exposed. Rotating keys regularly or right after a suspected compromise helps limit the damage. It takes a bit of extra setup, but it is definitely worth doing.

6.2.4. Avoid Insecure Random Number Generators

If you are using randomness for anything security-related, like session IDs or tokens, it needs to be truly unpredictable. Functions like `rand()` from the standard library are usually not enough because they are not designed for security. You should use cryptographically secure random functions, or there is a real risk someone could guess what is supposed to be random.

6.2.5. Don't Use Hardcoded Constants for Secrets

Developers sometimes hardcode things like salts, keys, or Initialization Vector (IVs), assuming it is safe as long as the code stays private. But if that code ever leaks or ends up on GitHub, that constant becomes a vulnerability. Secrets should be generated and stored securely outside the code. It is one of those things that seems minor until it is not.

6.3. Deployment and Infrastructure Hardening

6.3.1. Secure Default Configurations and Remove Unused Services

A lot of software and servers come with default configurations that are not secure, like open ports, default passwords, or even demo pages still running. If you deploy something

without changing those, you could be exposing more than you realize. It is best to review the settings and disable anything you are not actually using before it goes live.

6.3.2. Disable Directory Listing

If your server shows a directory listing when someone accesses a Uniform Resource Locator (URL), that is a problem. It exposes a list of files, including backup copies or old scripts you might have forgotten about. Unless there is a specific reason to keep it enabled, which is rare, directory listing should be turned off.

6.3.3. Verify Software Updates

When installing updates, especially from third-party sources, you should verify they have not been tampered with. This usually means checking cryptographic signatures or hashes. It is a small step, but it helps prevent supply chain attacks where malicious code is inserted before you even install it.

6.3.4. Use Container Security Best Practices

Containers are very useful, but they are not secure by default. You should not run them as root unless there is no other option. Use minimal base images, scan for known vulnerabilities, and include only what is actually needed in each container. If you are pulling images from external sources, try to sign and verify them. It is easy to overlook but important.

6.3.5. Keep Dependencies Up to Date

A lot of security issues come from old dependencies that stick around. If you are not keeping track of updates, you could be running code with known bugs for years without realizing it. Use tools that alert you when something is out of date or needs patching. It is one of those things that saves you trouble later on.

6.3.6. Maintain a Software Bill of Materials (SBOM)

A SBOM is a list of everything your application depends on, including third-party libraries and components. If a new vulnerability appears in one of them, an SBOM helps you quickly determine whether your software is affected. It might seem unexciting, but it is extremely useful when things go wrong, like with Log4Shell.

6.4. *Input and Output Handling Best Practices*

6.4.1. Validate All User Inputs to Prevent SQL Injection and XSS

This one is important. Any time your application accepts input, whether from forms, URLs, or headers, you need to validate it. Do not just check that it is present, but make sure it matches what you actually expect. SQL injection and XSS are still two of the most common attacks, mostly because developers trust input too much. You should not. Always assume the input is trying to break something.

6.4.2. Use Type Checks and Sanitization

It is not enough to just check that something was submitted, you also need to make sure it is the right type. If you are expecting a number, confirm it is a number. If it is a date, check the format. After that, sanitize it by removing or neutralizing anything that could be dangerous, like script tags or unexpected characters. Even if it seems harmless, it is safer to be strict from the start.

6.4.3. Canonicalize Inputs

Attackers often use encoding tricks to bypass filters. Instead of writing “../” in a file path, they might use “%2E%2E%2F” which means the same thing but looks different. Canonicalization is the process of converting input into a standard format before validating it. This helps ensure you do not miss anything just because it is written in a different way, and it catches attempts to sneak past basic checks. Here’s a simple example in Python that shows how to normalize and safely validate a user-provided file path in Listing 3:

Listing 3. Secure Path Canonicalization and Validation in Python.

```
1 import os
2 user_input = "../etc/passwd"
3 base_dir = "/app/data"
4 # Canonicalize the path to resolve all relative elements (e.g., "../")
5 full_path = os.path.realpath(os.path.join(base_dir, user_input))
6 # Validate that the canonicalized path starts with the trusted base
  directory.
7 # This prevents Path Traversal attacks.
8 if not full_path.startswith(base_dir):
9     raise ValueError("Invalid file path")
```

This code takes user input and uses `realpath` to resolve it into a clean, absolute path. Then it checks whether the resulting path stays within a trusted base directory. This prevents attackers from using tricks like `../` or encoded versions to escape into sensitive parts of the file system.

6.4.4. Validate File Uploads

Never allow users to upload files without proper checks. You should verify the file type, limit the size, and scan for malware if possible. Also, avoid storing uploaded files in public directories where they could be accidentally executed. A bad upload can quickly turn into remote code execution if you are not careful.

6.4.5. Use Schema Validation (JSON/XML)

If your app’s taking in JavaScript Object Notation (JSON) or XML from users or APIs, you need to make sure it sticks to a strict structure. Schema validation helps you catch requests that are missing required fields, have extra junk, or try to sneak in the wrong data types. Don’t just parse the data and assume it’s fine, check it first.

6.4.6. Prevent XXE (XML External Entity) Attacks

Some XML parsers allow input to reference external files or internal services. If that feature is not disabled, attackers can use it to read files on your server. If you are using XML, make sure XML External Entity (XXE) processing is turned off.

6.4.7. Sanitize Output Before Rendering

Just because input is clean does not mean the output is safe. If you display something like a user’s name on a page without proper escaping, it can still lead to XSS. You need to use the right kind of output encoding based on where the data is going, whether it is in HTML, JavaScript, URLs, or elsewhere. What works in one context might not work in another.

6.5. Secure Development Processes

6.5.1. Follow Secure Coding Guidelines (like OWASP or CERT)

You do not need to start from scratch. There are already well-written secure coding guidelines available. OWASP is one of the most widely used, and CERT is especially

helpful for languages like C and Java. It is easy to skip over this and just wing it, but clear standards help keep everyone on the same page and catch common issues early.

6.5.2. Use Static and Dynamic Analysis Tools

Static tools analyze your code without running it. They are good at catching hardcoded secrets, unsafe functions, and logic issues before anything goes live. Dynamic tools test the application while it is running, almost like stress-testing it from the outside. Both are useful, but not perfect. You still need manual testing and reviews, but these tools can catch issues you might otherwise miss.

6.5.3. Remove Debug Code Before Release

Leaving things like `console.log` statements, test endpoints, or even accidental backdoors in production happens more often than people admit, especially when teams are moving quickly. Before anything goes live, take the time to double-check and clean out any debug code. Some tools can help, but a quick manual review is usually enough.

6.5.4. Handle Exceptions Safely

When something goes wrong, do not let the app crash or display a full stack trace to the user. Handle exceptions properly: log them securely and show a generic error message that does not reveal sensitive information. Attackers will often use detailed error messages to learn how the system works.

6.5.5. Mask Sensitive Data in Logs

Logging is important, but it can backfire if you are not careful. Do not log plaintext passwords, tokens, credit card numbers, or any other sensitive data. If that kind of information really needs to be included, which it usually does not, make sure it is masked or encrypted. Logs should help debug issues, not give useful information to an attacker.

6.6. Design and Logic Security

6.6.1. Avoid Dynamic Code Execution (`eval`, `exec`, etc.)

Using `eval()` or similar functions is almost always a bad idea. It allows dynamic code execution, which might seem flexible, but if any part of the input can be influenced by a user, it creates serious risk. There is usually a safer way to achieve the same result, like using templates or input mapping. If dynamic execution is truly necessary, it should be sandboxed and tightly controlled. But avoid it unless there is absolutely no other option. Here is a simple example showing why `eval()` is dangerous, and how to replace it with a safer alternative in Listing 4:

Listing 4. Dangerous Dynamic Execution vs. Safe Input Mapping in Python.

```
1 # Dangerous: user controls code being executed
2 user_input = "os.system('rm -rf /')"
3 eval(user_input) # Can execute anything!
4
5 # Safer: controlled input mapping
6 commands = {
7     "status": lambda: "OK",
8     "ping": lambda: "PONG"
9 }
10 user_input = "ping"
11 result = commands.get(user_input, lambda: "Invalid command")()
```

The first example executes whatever string the user provides, which is extremely risky. The second example uses a safe mapping of allowed inputs to known functions. It

avoids the need for dynamic execution entirely, reducing the attack surface and making the behavior predictable.

6.6.2. Protect Critical Business Logic

Not all attacks rely on technical bugs. Some exploit flaws in how the application flows. For example, skipping steps in a checkout process or using a password reset link without confirming identity. These logic issues are easy to miss because the code works, but it behaves in the wrong order. You need to think through each step and ask, “What happens if someone does this differently?”.

6.6.3. Don’t Mix Trusted and Untrusted Data

Combining user input with internal data, especially when building commands, queries, or configuration files, is risky if that input has not been fully validated. Anything from outside should be treated as untrusted until it is properly cleaned up. Mixing unvalidated input with internal logic can lead to serious vulnerabilities, even if the rest of the code appears correct.

6.6.4. Watch Internal Flags and Loop Counters

Flags, counters, and other internal state variables might seem harmless, but if they are changed in the wrong place, they can cause subtle issues. For example, skipping a security check because a loop counter was modified too early. These bugs are easy to miss, so the logic should be clear, and critical values should be verified at the right time.

6.6.5. Enforce Strong Typing and Input Formats

In loosely typed languages like JavaScript or Python, you cannot assume inputs are what you expect. Someone might send a string when you are expecting a number, which can break logic or cause unexpected behavior. Always check both the type and the format, even if it seems obvious.

6.6.6. Validate Third-Party Code Assumptions

Just because a library is popular or well-documented does not mean it is secure for your specific use case. You still need to test how it handles edge cases, what kind of inputs it expects, and whether that matches how your application works. Many libraries do not validate input properly or make assumptions that might not apply to your setup. So do not trust them out of the box, check and monitor how they behave.

6.6.7. Set and Enforce CSP Headers

Content Security Policy (CSP) headers help control what your application is allowed to load and run in the browser. They can block inline scripts, restrict content sources, and prevent many XSS-related issues. While not a complete fix, CSP adds a strong layer of defense. Many developers skip it because the configuration can be tricky, but once it is set up, it can catch problems that might otherwise go unnoticed.

6.7. *Monitoring and Incident Response*

6.7.1. Log Security Events for Anomaly Detection

Logging is not just for debugging, it is a key part of your security setup, even if it does not always feel like it. You should log failed login attempts, role changes, password resets, and anything else that might indicate suspicious activity. Without those logs, you are essentially blind to what is happening. Logs should be detailed, structured (JSON is a good option), and stored securely, not just dumped in a folder no one checks.

6.7.2. Review Logs for Suspicious Activity

Logging is only useful if someone actually reviews it. You should have alerts in place for known issues, but also perform regular reviews, manually or through automation, to catch anything that slips through. Watch for unusual IP addresses, traffic spikes, or behavior that does not make sense. It is easy to overlook logs until something goes wrong.

6.7.3. Hide Error Details from Users

When something breaks, users should see a basic error message, not a full stack trace or detailed database error. Messages like that can give attackers too much insight into how the system works. Keep detailed information in your logs, and only show users what they need to see.

6.7.4. Turn Off Debugging in Production

Debug mode in production can leak too much information, like environment variables, internal paths, or even API keys. It is easy to overlook during testing, but you need to check your configuration before anything goes live. Ideally, debug features should be turned off by default and only enabled in a proper development environment.

6.7.5. Mask Sensitive Info in Logs

Never log passwords, credit card numbers, or full tokens, even if the logs are internal. If sensitive data absolutely needs to be included, make sure it is masked, such as showing only the last few digits. One bad log line can lead to a major data leak if you are not careful.

6.7.6. Track Rate Limiting and Abuse Patterns

Rate limiting is not just about slowing things down. It is also a simple way to detect brute-force attempts or other unusual activity. If someone is hitting your login endpoint 50 or 100 times in a minute, that is not normal and should raise a flag. You should track these events and watch for patterns over time to catch issues early, before they turn into something more serious.

7. Discussion

The synthesis of these secure coding practices reveals a critical discrepancy between developer intent and security outcomes. While practitioners generally acknowledge the importance of security, the fragmented nature of existing guidelines—spanning disparate standards from OWASP, NIST, and CERT—imposes a significant cognitive load that impedes consistent implementation [11]. This section evaluates the performance of the proposed unified framework in terms of its robustness, efficiency, and scholarly implications.

7.1. Comparative Robustness and Coverage

In terms of **robustness**, the proposed framework offers a more comprehensive defensive posture compared to single-source checklists. While the OWASP Top Ten provides high accuracy in identifying the most prevalent web-based risks, it often neglects low-level resource management or long-term infrastructure security. Our Generative AI-assisted mapping exercise demonstrates that a unified synthesis achieves a 94% coverage rate across the CWE Top 25, addressing gaps in business logic and deployment configuration that are frequently omitted from reactive vulnerability lists. Furthermore, when compared to automated Static Application Security Testing (SAST) tools, this framework demonstrates superior **contextual efficiency**. While SAST tools are highly efficient at identifying syntax-level flaws, such as the use of an insecure hash, they lack the robustness to detect architectural logic flaws, such as the omission of a critical authorization check in a complex

business workflow. By providing developers with a proactive “Security-by-Design” mental model, this framework mitigates the “false sense of security” often induced by purely tool-centric approaches.

7.2. Theoretical and Practical Implications

The **theoretical implication** of this work is the shift from a “vulnerability-centric” paradigm to a “practice-centric” one. Historically, secure coding has been taught as a series of patches for known exploits; however, this research proposes a structured taxonomy that treats security as a functional requirement of the Software Development Lifecycle (SDLC).

From a **practical standpoint**, the results suggest that the “Actionability Gap” is best bridged by shifting the security burden from individual developers to secure-by-default framework configurations. Furthermore, leveraging the framework’s eight domains to build smarter IDE integration can provide real-time, context-aware guidance during the coding process. Ultimately, standardizing terminology across the 35+ synthesized sources reduces the “responsibility diffusion” often seen in large teams, where members are otherwise unsure which disparate standard to follow.

8. Conclusions

This research addressed the critical problem of knowledge fragmentation in software security by synthesizing over 35 authoritative sources into a unified, eight-domain framework. By transitioning from a narrative description to a structured synthesis, we have provided a roadmap that aligns disparate standards from OWASP, CERT, and NIST into a single, developer-centric resource. Our results indicate that such a unified approach provides a more robust defense against common vulnerabilities (CWEs) than existing siloed guidelines, particularly in areas of business logic and infrastructure security. Ultimately, this work demonstrates that secure coding must be treated as a habituated design practice rather than a reactive checklist. By integrating these synthesized practices into the early stages of the SDLC, development teams can systematically reduce the attack surface of modern applications.

9. Limitations and Future Work

Despite the robustness of the synthesized framework, the rapid evolution of AI-generated code introduces a speed of development that may outpace manual review cycles. Additionally, while the framework is language-agnostic, its application in emerging paradigms like serverless computing or AI model training requires further empirical validation.

Future work will prioritize empirical codebase analysis by implementing the framework on large-scale open-source projects to quantitatively measure which domains are most frequently neglected. We also intend to explore tooling and automation through the development of lightweight IDE extensions that highlight these practices in real-time, effectively bridging the gap between documentation and implementation. Finally, a sociotechnical evaluation will be conducted to investigate how team-level factors, such as culture and deadline pressure, affect adherence to these domains, moving toward a more holistic understanding of software security in professional environments.

Author Contributions: Conceptualization, A.A., M.K.S., G.A., A.K., L.A. and M.U.K.; methodology, M.U.K., M.K.S., M.J.H.F. and H.S.; investigation, M.K.S., A.A., G.A., A.K. and L.A.; writing—original draft preparation, M.K.S., A.A., G.A., A.K. and L.A.; writing—review and editing, M.U.K., M.K.S., M.J.H.F. and H.S.; supervision, M.U.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data generated during this study are presented within the article (see Table 9). The codebase used for the framework validation is the OWASP Juice Shop, an open-source project publicly available at: <https://owasp.org/www-project-juice-shop/> (accessed on 16 February 2026). All other sources used for the synthesis are cited in the References section.

Acknowledgments: The authors would like to acknowledge Alfaisal University for its support.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Alenezi, M.; Zarour, M. On the relationship between software complexity and security. *arXiv* **2020**, arXiv:2002.07135.
- Mulahuwaish, A.; Qolomany, B.; Gyorick, K.; Abdo, J.B.; Aledhari, M.; Qadir, J.; Carley, K.; Al-Fuqaha, A. A survey of social cybersecurity: Techniques for attack detection, evaluations, challenges, and future prospects. *Comput. Hum. Behav. Rep.* **2025**, *18*, 100668. [\[CrossRef\]](#)
- Siam, M.K.; Saha, B.; Hasan, M.M.; Hossain Faruk, M.J.; Anjum, N.; Tahora, S.; Siddika, A.; Shahriar, H. Securing Decentralized Ecosystems: A Comprehensive Systematic Review of Blockchain Vulnerabilities, Attacks, and Countermeasures and Mitigation Strategies. *Future Internet* **2025**, *17*, 183. [\[CrossRef\]](#)
- Fredj, O.B.; Cheikhrouhou, O.; Krichen, M.; Hamam, H.; Derhab, A. An OWASP top ten driven survey on web application protection methods. In *Proceedings of the International Conference on Risks and Security of Internet and Systems*; Springer: Cham, Switzerland, 2020; pp. 235–252.
- Kioskli, K.; Grigoriou, E.; Islam, S.; Yiorkas, A.M.; Christofi, L.; Mouratidis, H. A risk and conformity assessment framework to ensure security and resilience of healthcare systems and medical supply chain. *Int. J. Inf. Secur.* **2025**, *24*, 90. [\[CrossRef\]](#)
- Vassallo, C.; Panichella, S.; Palomba, F.; Proksch, S.; Gall, H.C.; Zaidman, A. How developers engage with static analysis tools in different contexts. *Empir. Softw. Eng.* **2020**, *25*, 1419–1457. [\[CrossRef\]](#)
- Wendt, D.W. Regulations, Standards, and Frameworks. In *AI Strategy and Security: A Roadmap for Secure, Responsible, and Resilient AI Adoption*; Apress: New York, NY, USA, 2025; pp. 137–157.
- Acar, Y.; Stransky, C.; Wermke, D.; Weir, C.; Mazurek, M.L.; Fahl, S. Developers need support, too: A survey of security advice for software developers. In *Proceedings of the 2017 IEEE Cybersecurity Development (SecDev)*, Cambridge, MA, USA, 24–26 September 2017.
- Hove, S. Exploring Factors Influencing the Integration of Secure Code Development Protocols Within the Software Development Lifecycle. Ph.D. Thesis, Capella University, Minneapolis, MN, USA, 2025.
- Yusop, M.I.M.; Kamarudin, N.H.; Suhaimi, N.H.S.; Hasan, M.K. Advancing passwordless authentication: A systematic review of methods, challenges, and future directions for secure user identity. *IEEE Access* **2025**, *13*, 13919–13943. [\[CrossRef\]](#)
- Altulaihan, E.A.; Alismail, A.; Frikha, M. A survey on web application penetration testing. *Electronics* **2023**, *12*, 1229. [\[CrossRef\]](#)
- Newhouse, B.; Souppaya, M.; Cooper, D.; Polk, T.; Barker, W.; Scarfone, K.; Kent, J.; Sexton, J.; Dimond, M.; Klosterman, J.; et al. Addressing Visibility Challenges with TLS 1.3 within the Enterprise High-Level Document. *NIST Spec. Publ.* **2025**, *1800*, 37.
- Rauf, I.; Lopez, T.; Sharp, H.; Petre, M.; Tun, T.; Levine, M.; Towse, J.; van der Linden, D.; Rashid, A.; Nuseibeh, B. Influences of developers' perspectives on their engagement with security in code. In *Proceedings of the 15th International Conference on Cooperative and Human Aspects of Software Engineering*, Pittsburgh, PA, USA, 18–19 May 2022.
- Gasiba, T.E.; Pereira, J.M.; de Oliveira, L.C.; Murta, D.L.P. Is secure coding education in the industry needed? In *Proceedings of the 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, Madrid, Spain, 25–28 May 2021. [\[CrossRef\]](#)
- Yang, Y.; Wu, Y.; Pattabiraman, K.; Wang, L.; Li, Y. How far have we come in detecting anomalies in distributed systems? An empirical study with a statement-level fault injection method. In *Proceedings of the IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, 12–15 October 2020. [\[CrossRef\]](#)
- Lopez, T.; Sharp, H.; Tun, T.; Bandara, A.; Levine, M.; Nuseibeh, B. "Hopefully We Are Mostly Secure": Views on Secure Code in Professional Practice. In *Proceedings of the 2019 IEEE/ACM 12th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, Montreal, QC, Canada, 27 May 2019. [\[CrossRef\]](#)
- Oh, S.; Srivastava, A.; Mirhosseini, A.; Yoon, J.; Roesner, F.; Kohno, T. Poisoned ChatGPT finds work for idle hands: Exploring developers' coding practices with insecure suggestions from poisoned AI models. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, USA, 19–23 May 2024. [\[CrossRef\]](#)
- Perry, N.; Suresh, S.; Nair, A.; Zorn, B.; Livshits, B.; Tarlow, D. Do users write more insecure code with AI assistants? *arXiv* **2022**, arXiv:2211.03622. [\[CrossRef\]](#)

19. Razzaq, A.; Buckley, J.; Lai, Q.; Yu, T.; Botterweck, G. A systematic literature review on the influence of enhanced developer experience on developers' productivity: Factors, practices, and recommendations. *ACM Comput. Surv.* **2024**, *57*, 1–46. [\[CrossRef\]](#)
20. Pearce, H.; Ahmad, B.; Tan, B.; Dolan-Gavitt, B.; Karri, R. Asleep at the keyboard? Assessing the security of github copilot's code contributions. *Commun. ACM* **2025**, *68*, 96–105. [\[CrossRef\]](#)
21. Perry, N.; Srivastava, M.; Kumar, D.; Boneh, D. Do users write more insecure code with ai assistants? In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Copenhagen, Denmark, 26–30 November 2023.
22. Aghakhani, H.; Dai, W.; Manoel, A.; Fernandes, X.; Kharkar, A.; Kruegel, C.; Vigna, G.; Evans, D.; Zorn, B.; Sim, R. TrojanPuzzle: Covertly Poisoning Code-Suggestion Models. *arXiv* **2023**, arXiv:2301.02344. [\[CrossRef\]](#)
23. Chung, S.; Endicott-Popovsky, B. Software reengineering based security teaching. In Proceedings of the International Conference on Cybernetics and Information Technologies, Systems and Applications (CITSA), Orlando, FL, USA, 29 June–2 July 2010.
24. Nembhard, F.D.; Carvalho, M.M.; Eskridge, T.C. Towards the application of recommender systems to secure coding. *EURASIP J. Inf. Secur.* **2019**, *2019*, 9. [\[CrossRef\]](#)
25. Ryan, I.; Roedig, U.; Stol, K.J. Training developers to code securely: Theory and practice. In Proceedings of the ACM/IEEE International Conference on Engineering Secure Software and Systems (EnCyCriS), Lisbon, Portugal, 15 April 2024. [\[CrossRef\]](#)
26. Meng, N.; Nagappan, M.; Bird, C.; Zimmermann, T. Secure coding practices in Java: Challenges and vulnerabilities. In Proceedings of the 40th International Conference on Software Engineering (ICSE), Gothenburg, Sweden, 27 May–3 June 2018. [\[CrossRef\]](#)
27. Votipka, D.; Rabinovitch, A.; Redmiles, E.; Mazurek, P.; Mazurek, M.L. Understanding security mistakes developers make: Qualitative analysis from Build It, Break It, Fix It. In Proceedings of the USENIX Security Symposium, Boston, MA, USA, 12–14 August 2020.
28. Grassi, P.A.; Garcia, M.E.; Fenton, J.L. *Digital Identity Guidelines: Authentication and Lifecycle Management*; Technical Report SP 800-63B; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2017. [\[CrossRef\]](#)
29. MITRE. CWE-521: Weak Password Requirements. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/521.html> (accessed on 16 February 2026).
30. MITRE. CWE-798: Use of Hard-Coded Credentials. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/798.html> (accessed on 16 February 2026).
31. MITRE. CWE-307: Improper Restriction of Excessive Authentication Attempts. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/307.html> (accessed on 16 February 2026).
32. MITRE. CWE-1188: Insecure Default Initialization of Resource. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1188.html> (accessed on 16 February 2026).
33. Black, P.; Guttman, B.; Okun, V. *Guidelines on Minimum Standards for Developer Verification of Software*; NIST Interagency or Internal Report (IR) 8397; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2021.
34. Rahayu, S.B.; Ahmad, A.R.; Rashid, Z. Defensive programming: Developing a web application with secure coding practices. *AIP Conf. Proc.* **2022**, *2617*, 050006. [\[CrossRef\]](#)
35. Loncar, K.; Borsic, M.; Dukic, M. Secure coding guidelines and standards. In Proceedings of the 35th DAAAM International Symposium on Intelligent Manufacturing and Automation, Vienna, Austria, 24–25 October 2024. [\[CrossRef\]](#)
36. Weir, C.; Ozment, A.; Seaman, C.B. Light-touch interventions to improve software development security. In Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 20–24 May 2018. [\[CrossRef\]](#)
37. Nguyen, D.; Kayes, A.M.M.; Babar, M.A.; Cavallaro, L.; Lo, D. A stitch in time: Supporting Android developers in writing secure code. In Proceedings of the ACM Conference on Computer and Communications Security (CCS), Dallas, TX, USA, 30 October 3–November 2017. [\[CrossRef\]](#)
38. Zeng, M.; Zhu, F. Secure coding in five steps. *J. Cybersecur. Educ. Res. Pract.* **2021**, *2021*, 5. [\[CrossRef\]](#)
39. dos Santos, R.M.; Gerosa, M.A. Impacts of coding practices on readability. In Proceedings of the 26th Conference on Program Comprehension, Gothenburg, Sweden, 28–29 May 2018. [\[CrossRef\]](#)
40. Thatikonda, V.; Mudunuri, H.R.V. Writing secure code in the digital age: Preventing common vulnerabilities. *Int. J. Comput. Appl.* **2023**, *185*, 48–51. [\[CrossRef\]](#)
41. MITRE. CWE-320: Use of Hard-Coded Cryptographic Key. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/320.html> (accessed on 16 February 2026).
42. MITRE. CWE-547: Use of Hard-Coded, Security-Relevant Constants. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/547.html> (accessed on 16 February 2026).
43. MITRE. CWE-1102: Reliance on Machine-Dependent Data Representation. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1102.html> (accessed on 16 February 2026).
44. MITRE. CWE-1127: Compilation with Insufficient Warnings or Errors. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1127.html> (accessed on 16 February 2026).

45. MITRE. CWE-330: Use of Insufficiently Random Values. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/330.html> (accessed on 16 February 2026).
46. Sheta, S.V. Implementing secure and efficient code in system software development. *Int. J. Inf. Technol. Manag. Inf. Syst.* **2024**, *15*, 34–46. [CrossRef]
47. MITRE. CWE-478: Missing Default Case in Multiple Condition Expression. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/478.html> (accessed on 16 February 2026).
48. MITRE. CWE-1104: Use of Unmaintained Third Party Components. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1104.html> (accessed on 16 February 2026).
49. Chowdhury, M.N.U.R.; Chowdhury, F.H.; Kayes, A.S.M. AI-driven secure coding: Revolutionizing source code defense. In Proceedings of the IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS), Lucknow, India, 12–13 September 2024. [CrossRef]
50. MITRE. CWE-605: Multiple Binds to the Same Port. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/605.html> (accessed on 16 February 2026).
51. MITRE. CWE-1101: Insufficient Isolation of System-Dependent Functions. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1101.html> (accessed on 16 February 2026).
52. NIST National Vulnerability Database. CVE-2021-22005 Detail. 2021. Available online: <https://nvd.nist.gov/vuln/detail/cve-2021-22005> (accessed on 16 February 2026).
53. MITRE. CWE-548: Exposure of Information Through Directory Listing. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/548.html> (accessed on 16 February 2026).
54. MITRE. CWE-487: Exposure of System Data to an Unauthorized Control Sphere. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/487.html> (accessed on 16 February 2026).
55. MITRE. CWE-358: Improperly Unlocked Resource. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/358.html> (accessed on 16 February 2026).
56. MITRE. CWE-1235: Data Element Aggregating an Excessively Large Number of Non-Primitive Elements. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1235.html> (accessed on 16 February 2026).
57. MITRE. CWE-119: Improper Restriction of Operations Within the Bounds of a Memory Buffer. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/119.html> (accessed on 16 February 2026).
58. MITRE. CWE-404: Improper Resource Shutdown or Release. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/404.html> (accessed on 16 February 2026).
59. White, G. *Secure Coding Practices, Tools, and Processes*; Technical Report LLNL-CONF-671591; Lawrence Livermore National Laboratory: Livermore, CA, USA, 2015.
60. Turner, S. Security Vulnerabilities of the Top Ten Programming Languages: C, Java, C++, Objective-C, C#, PHP, Visual Basic, Python, Perl, and Ruby. Academic and Business Research Institute (AABRI). Available online: <https://www.aabri.com/LV2013Manuscripts/LV13090.pdf> (accessed on 16 February 2026).
61. Anis, A.; Zulkernine, M.; Iqbal, S.; Liem, C.; Chambers, C. Securing Web Applications with Secure Coding Practices and Integrity Verification. In Proceedings of the 2018 IEEE 16th International Conference on Dependable, Autonomic and Secure Computing, 16th International Conference on Pervasive Intelligence and Computing, 4th International Conference on Big Data Intelligence and Computing, and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), Athens, Greece, 12–15 August 2018. [CrossRef]
62. Ryan, I.; Roedig, U.; Stol, K.J. Measuring Secure Coding Practice and Culture: A Finger Pointing at the Moon is not the Moon. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, VIC, Australia, 14–20 May 2023. [CrossRef]
63. Smith, J.M.; Schuchard, M. Routing Around Congestion: Defeating DDoS Attacks and Adverse Network Conditions via Reactive BGP Routing. In Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 21–23 May 2018. [CrossRef]
64. MITRE. CWE-489: Active Debug Code. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/489.html> (accessed on 16 February 2026).
65. MITRE. CWE-248: Uncaught Exception. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/248.html> (accessed on 16 February 2026).
66. MITRE. CWE-209: Generation of Error Message Containing Sensitive Information. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/209.html> (accessed on 16 February 2026).
67. MITRE. CWE-532: Insertion of Sensitive Information into Log File. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/532.html> (accessed on 16 February 2026).
68. MITRE. CWE-215: Insertion of Sensitive Information into Debugging Output. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/215.html> (accessed on 16 February 2026).

69. Jakimoski, K.; Stefanovska, Z.; Stefanovski, V. Optimization of Secure Coding Practices in SDLC as Part of Cybersecurity Framework. *J. Comput. Sci. Res.* **2022**, *4*, 31–41. [CrossRef]
70. MITRE. CWE-807: Reliance on Untrusted Inputs in a Security Decision. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/807> (accessed on 16 February 2026).
71. MITRE. CWE-1092: Use of Same Invokable Control Element in Multiple Architectural Layers. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1092.html> (accessed on 16 February 2026).
72. MITRE. CWE-1099: Missing Handler for Support of Multiple International Standards. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1099.html> (accessed on 16 February 2026).
73. MITRE. CWE-1116: Inaccurate Comments. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1116.html> (accessed on 16 February 2026).
74. MITRE. CWE-502: Deserialization of Untrusted Data. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/502.html> (accessed on 16 February 2026).
75. MITRE. CWE-611: Improper Restriction of XML External Entity Reference. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/611.html> (accessed on 16 February 2026).
76. MITRE. CWE-20: Improper Input Validation. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/20.html> (accessed on 16 February 2026).
77. MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/770.html> (accessed on 16 February 2026).
78. Murphy, T. Security Challenges in the 21st Century Global Commons. *Yale J. Int. Aff.* **2010**, *5*, 28.
79. MITRE. CWE-1117: Inclusion of Undocumented Features or APIs. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1117.html> (accessed on 16 February 2026).
80. MITRE. CWE-840: Improper Neutralization of Delimiters in Controls in a Business Flow. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/840.html> (accessed on 16 February 2026).
81. MITRE. CWE-1021: Improper Restriction of Rendered UI Layers or Frames. Common Weakness Enumeration. 2023. Available online: <https://cwe.mitre.org/data/definitions/1021.html> (accessed on 16 February 2026).
82. Foundation, O. OWASP Juice Shop—The Most Insecure Web Application for Security Training. 2026. Available online: <https://owasp.org/www-project-juice-shop/> (accessed on 16 February 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.